

Inigo Quilez :: articles :: pop corn images - 1999



I first read about the so called "Pop Corn" images in a website by Cliff Pickover some time in the later 90s. The idea behind them was to plot the evolution of a dynamical system like

$$\mathbf{p}'(t) = \mathbf{v}(\mathbf{p})$$

where $\mathbf{p}$ is a point and $\mathbf{v}$ is a stationary velocity field. To make such a simulation in a computer, the simplest way is to code some simple Euler integrator which will simulate the derivative with some differences and a small delta time. In two dimensions, if we make $\mathbf{p} = (x, y)$ and $\mathbf{v} = (f, g)$, then that would look like this:

$$x_{n+1} = x_n + \lambda \cdot f(x, y)$$

$$y_{n+1} = y_n + \lambda \cdot g(x, y)$$

where λ is the time step value (it should be something small). Now it's up to you to choose nice formulas for $f(\mathbf{p})$ and $g(\mathbf{p})$. The original Pickover's formulas were trigonometric functions, but you can use anything you like. In my experiments of 1999 I used the original Pickover's formulas. The result made it into the 64 kb demo called *rare*. Later in 2006 I made the following video:

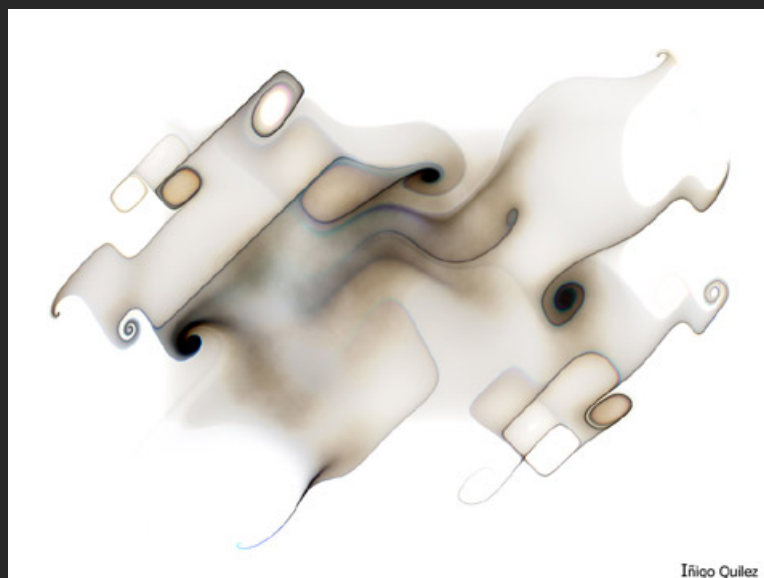


Here I animated $f(\mathbf{p})$ and $g(\mathbf{p})$ over time, using slightly different formulas to produce some nice fluid-like shapes:

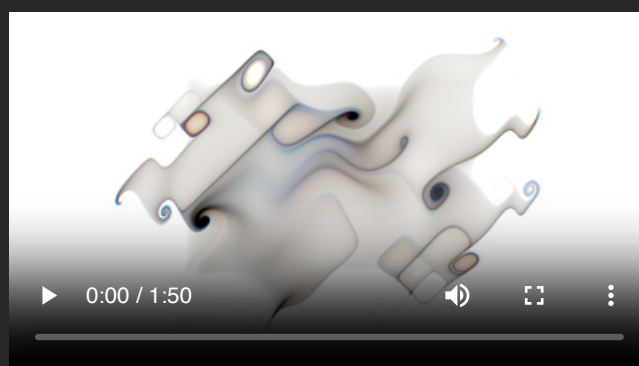
$$f(x, y) = \cos(t_0 + y + \cos(t_1 + \pi x))$$

$$g(x, y) = \cos(t_2 + x + \cos(t_3 + \pi y))$$

The t_i parameters are linearly time varying values that produce the actual animation.



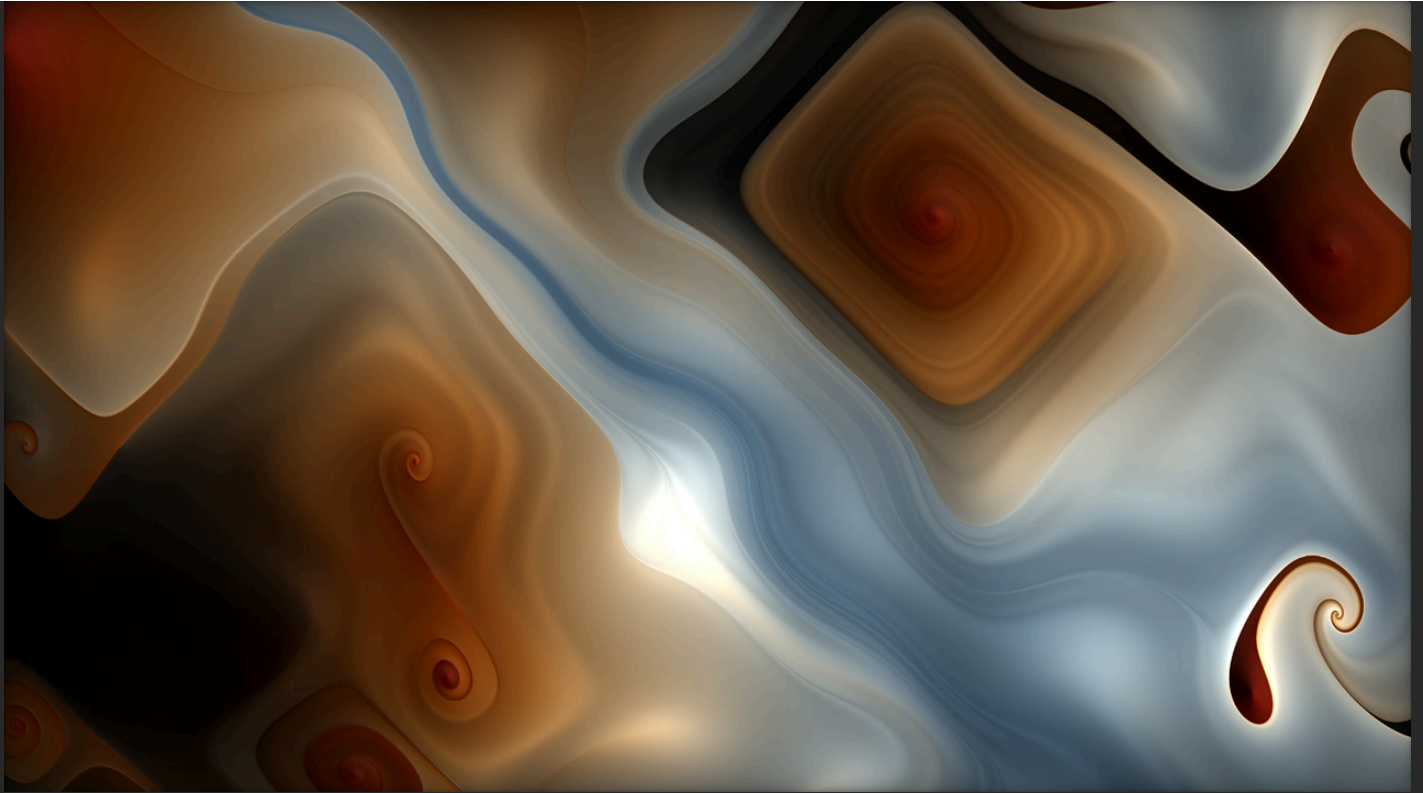
Enmpperaltta - a popcorn image (click to enlarge)



Variations - a popcorn video (click to play)

The way the video was rendered was this - just like with IFS methods, I picked a random point p in the plane and iterated it with the above formulas. This produced an orbit over space. I track the density of each pixel of the plane by counting how many times an iterated point landed in it. After performing some billion iterations, the image was noise-free enough, and it was interpreted as a density that I used to index into a color palette. In my case, what I did instead was to compute the density three times with slightly different parameters, and assign the resulting densities to the red, green and blue channels of the image.

But you can also use very different rendering algorithms. For example, the image below doesn't do a density calculation, and instead iterates the formulas for a different initial point p that is at the center of each pixel, and then computes the distance traveled by p as a result of the iterations. The image you get is this, and the code you can find it here: <https://www.shadertoy.com/view/Mdl3RH>:



inigo quilez - learning computer graphics since 1994